Parking Charge Strategy

Master of Science

Information & Communication Technology

Software Design & Programming

Eduardo Huamaní Quispe

University of Denver University College

Abril 17, 2025

Faculty: Nathan Braun, MBA

Director: Cathie Wilson, MS

Dean: Michael J. McGuire, MLS

Implementing Parking Charge Strategy

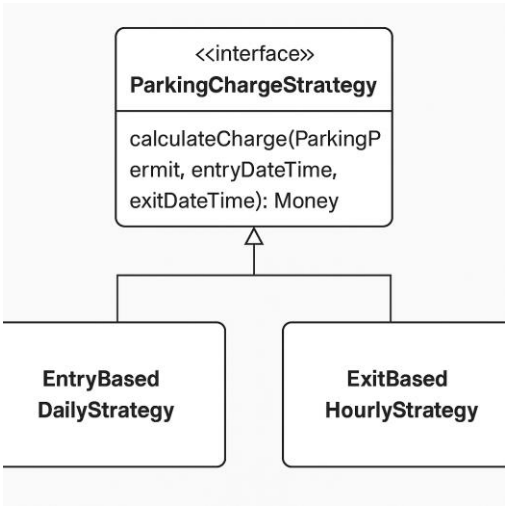
The University of Denver parking systems, has as one of its main functions to calculate the fee a customer must pay when they leave a parking lot. Pricing calculation rules can vary based on different considerations such as daily rates or hourly charges. It's necessary to make our implementation flexible by using the Strategy design pattern.

The ultimate purpose of this implementation is to allow different pricing strategies to be easily plugged into the system. Each strategy needs to account for the time the vehicle spent parked and the type of car, which may qualify for discounts.

In this document, we will walk through the design and implementation of this feature.

Design

We defined an interface called ParkingChargeStrategy with 2 classes implementing that interface as showed in the image bellow.



The interface defines the following method

Money calculateCharge(ParkingPermit permit, LocalDateTime entryDateTime, LocalDateTime exitDateTime);

This method takes in a ParkingPermit (which provides access to the associated Customer and their Car), along with the entry and exit times. The returning value is a Money object representing the fee for using the parking lot.

To provide different ways of calculating the fee, we created two classes that implement the strategy interface, EntryBasedDailyStrategy and ExitBasedHourlyStrategy.

Based on the task instructions. Each class encapsulates a distinct pricing policy.

EntryBasedDailyStrategy

This strategy charges customers based on the number of calendar days between the entry and exit time. We assume that once a car enters the lot, a full day is charged—even if the stay is less than 24 hours.

The implementation calculates the number of days between entryDateTime.toLocalDate() and exitDateTime.toLocalDate(), adds one (since partial days are billed as full), and then multiplies the result by a base rate. We also check the type of car (COMPACT or SUV) and apply a discount of 20% on COMPACT cars.

This strategy works well for lots that charge a flat fee per day and want to discourage short-term parking.

ExitBasedHourlyStrategy

This strategy calculates the fee based on the exact number of hours the car was parked. It rounds up partial hours and multiplies by a fixed hourly rate.

This implementation is more precise and fair for customers who park for just a few hours. As with the daily strategy, we apply a discount based on the car type.

This approach is more for students, professors, and others that uses the parking lot such for some hours a day. It can be parttime students or workers.

Code Structure

The main classes involved are:

- ParkingChargeStrategy: the common interface
- EntryBasedDailyStrategy: calculates based on days
- ExitBasedHourlyStrategy: calculates based on hours
- Money: an immutable class used for currency operations
- ParkingPermit: provides access to the car and customer
- Car: has a CarType enum to determine any applicable discounts

The advantage of using the Strategy pattern (Interfaces) is that we can easily swap one strategy for another, or even configure them dynamically at runtime. The system stays open to extension without needing to modify the rest of the parking logic.

Improvements

We could improve this design further by externalizing the rate configurations to a database or configuration file. That way, we wouldn't need to hard-code the base rates or discount values.

We could also introduce more pricing strategies in the future, such as:

- Time of day (e.g., prime time surchage)
- Special days (e.g., graduation)

To support this, we could consider using a Factory pattern to choose the appropriate strategy based on context.

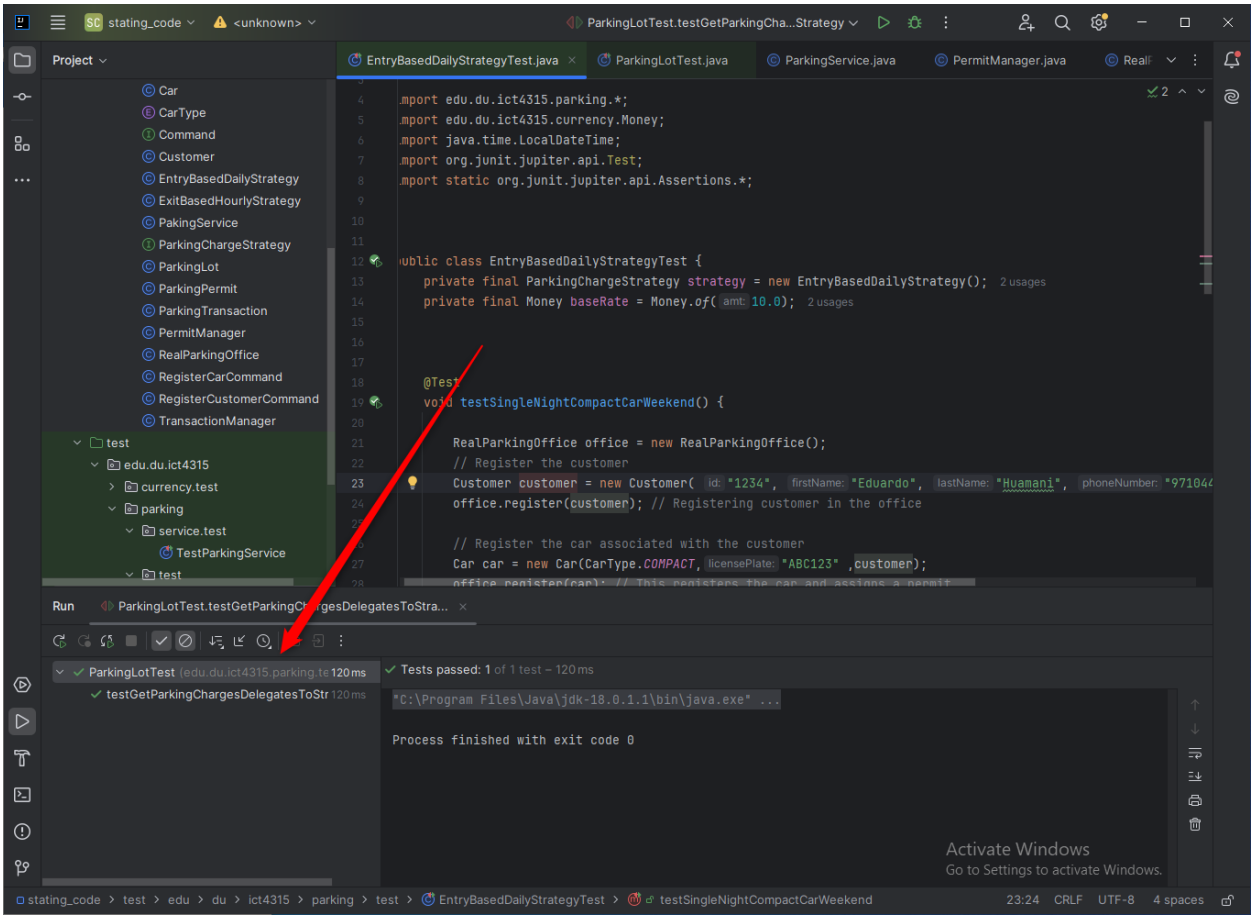
Difficulties

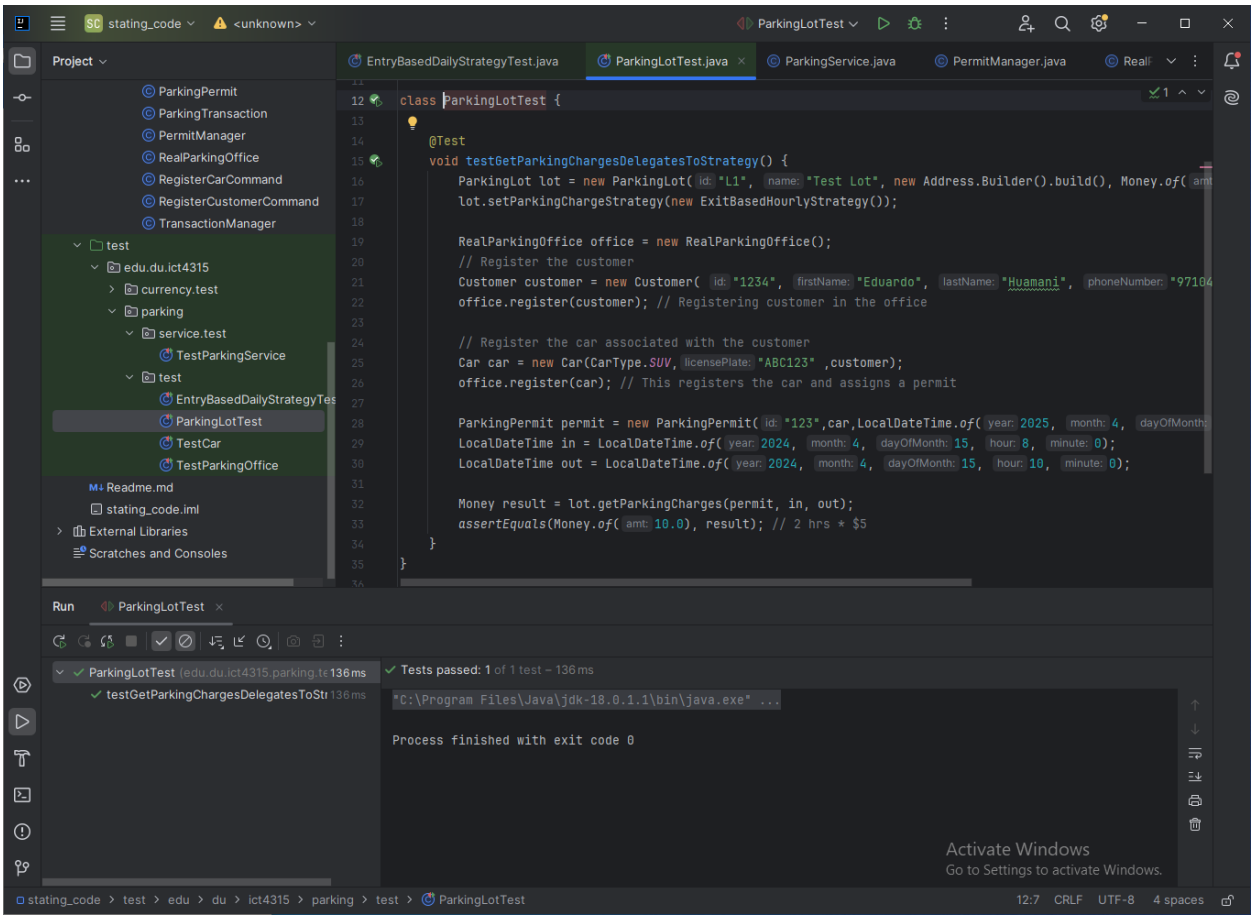
One of the tricky parts of this implementation was ensuring time calculations were precise, especially when rounding hours or converting between LocalDateTime and LocalDate. Care was needed to handle edge cases such as cars that park just before midnight or stay across multiple days.

Another challenge was balancing flexibility with simplicity—ensuring that our design allowed for future growth without overengineering the current solution.

Testing

We wrote unit tests for both EntryBasedDailyStrategy and ExitBasedHourlyStrategy. Since Money is a value class, we did not need to mock or stub it—just verified that the returned amounts matched expectations. The unit test results are showed in the images bellow.





Each test case included scenarios for SUV or COMPACT cars (which get a discount), and different durations of stay. We also tested borderline cases like staying for exactly 24 hours or just a few minutes past a day.

By isolating the strategy implementations, testing was straightforward and focused.

Conclusion

This was an interesting problem that allowed us to apply the Strategy design pattern(Interfaces) to a real-world scenario. By designing flexible pricing strategies, we set up our parking system to adapt to future business requirements without rewriting core logic. We ended up with a modular, testable solution that’s both practical and extendable.